
Q1.

An instruction for the ATMega32u2 is shown in Table 1.

Table 1: BRNE — branch if not equal — opcode specification

Syntax	Operands	Operation
`brne k`	$-64 \leq k \leq 63$	Result of CP, CPI, SUB, or SUBI $\neq 0$?
Control registers	Opcode	Flags
PC $\leftarrow$ PC + k + 1 (if Z = 0) PC $\leftarrow$ PC + 1 (if Z = 1)	1111 01kk kkkk k001	None

For the instruction 'branch if not equal':

(a) State what type/class of instruction it is.

> **Solution:** Branching (as opposed to data transfer or arithmetic).

(b) What is the binary sequence encoded by the hexadecimal `0xF4A1`?

> **Solution:** `0b1111 0100 1010 0001`

(c) Assuming that k is represented as a 7-bit two's complement integer, what is the value of k if the specific opcode sequence is `0xF4A1` ?

> **Solution:** From looking at the corresponding bits in the opcode that align with 'k', the 7-bit two's complement number (k) is therefore: `0b0010100` = $2^2 + 2^4 = 20$.

Another AVR instruction is shown in Table 2

Table 2: CPI — Compare immediate — opcode. Note: d has an offset of +16

Syntax	Operands	Operation
`cpi RD,K`	$16 \leq d \leq 31$, $0 \leq K \leq 255$	Rd - K
Control registers	Opcode	Flags
PC $\leftarrow$ PC + 1	0011 KKKK dddd KKKK	H,S,V,N,Z,C

(d) What does immediate addressing mean?

> **Solution:** Immediate addressing means a value is stored within the opcode. In the case of `cpi`, it means a register will be compared against a constant value defined in the actual opcode of the program.

PC	opcode (hex)	instruction
...		
0x0008:	3c 20	cpi rXX 0xXX;
0x0009:	f4 a1	brne, 20
...		

Register	Value
...	
R16	01
R17	00
R18	c0
R19	1a
R20	3c
R21	c2
R22	00
R23	f0
...	

(e) What is the value of `PC` after the following two instructions?

Solution: There are a few steps that we need to do to determine the solution.

From part c) we know PC will increase by 21 (k = `0b0010100`, PC ← PC + k + 1) if the register and the immediately addressed value in the `cpi` instruction are not equal.

For the comparison value, we need to determine a) which register is being compared, and b) the value K stored within the opcode.

For a) the register, this is equal to the unsigned binary number within d, plus 16. This can be almost directly read from the opcode, as the third nibble encodes d. The third nibble is equal to `0x2`, and thus we are comparing with register R18.

Similarly, we can read the comparison constant (K) directly from the opcode, as it is encoded in the second and fourth nibbles. Thus, K = `0xc0`.

The `CPI` instruction will therefore subtract `0xc0` from the value in R18, which is also `0xc0`. As the minuend and the subtrahend are equal, the result is zero (`0xc0 - 0xc` = 0), and the Z flag will be raised.

`BRNE` is the 'branch if not equal', and seeing as they are equal, it will not branch, and instead PC will increment per usual: PC ← PC + 1, so the PC will be `0x000a`.

Q2.

Consider the call instruction, shown in Table 3

Table 3: `call` instruction

Syntax	Operands	Operation
`call k`	$0 \leq k \leq 64K$	STACK ← PC + 2
Control registers	Opcode	Flags
PC ← k SP ← SP - 2	1001 010k kkkk 111k kkkk kkkk kkkk kkkk	None

(a) Explain what function changing the value of `PC` has.

Solution: The program counter — `PC` — is the pointer to the next instruction to be run. By changing the address to some arbitrary value (`k`), a function or some other code snippet at any memory location is able to be run.

(b) Explain why a call instruction is necessary, as opposed to a `brxx` branching instruction.

Solution: Branching instructions are relative to the current value of `PC`, and can only change the PC by 64 in either direction. This does not give access to all 16k possible instructions in the ATMega32u2, for example, and so changing the number to an arbitrary address is necessary.

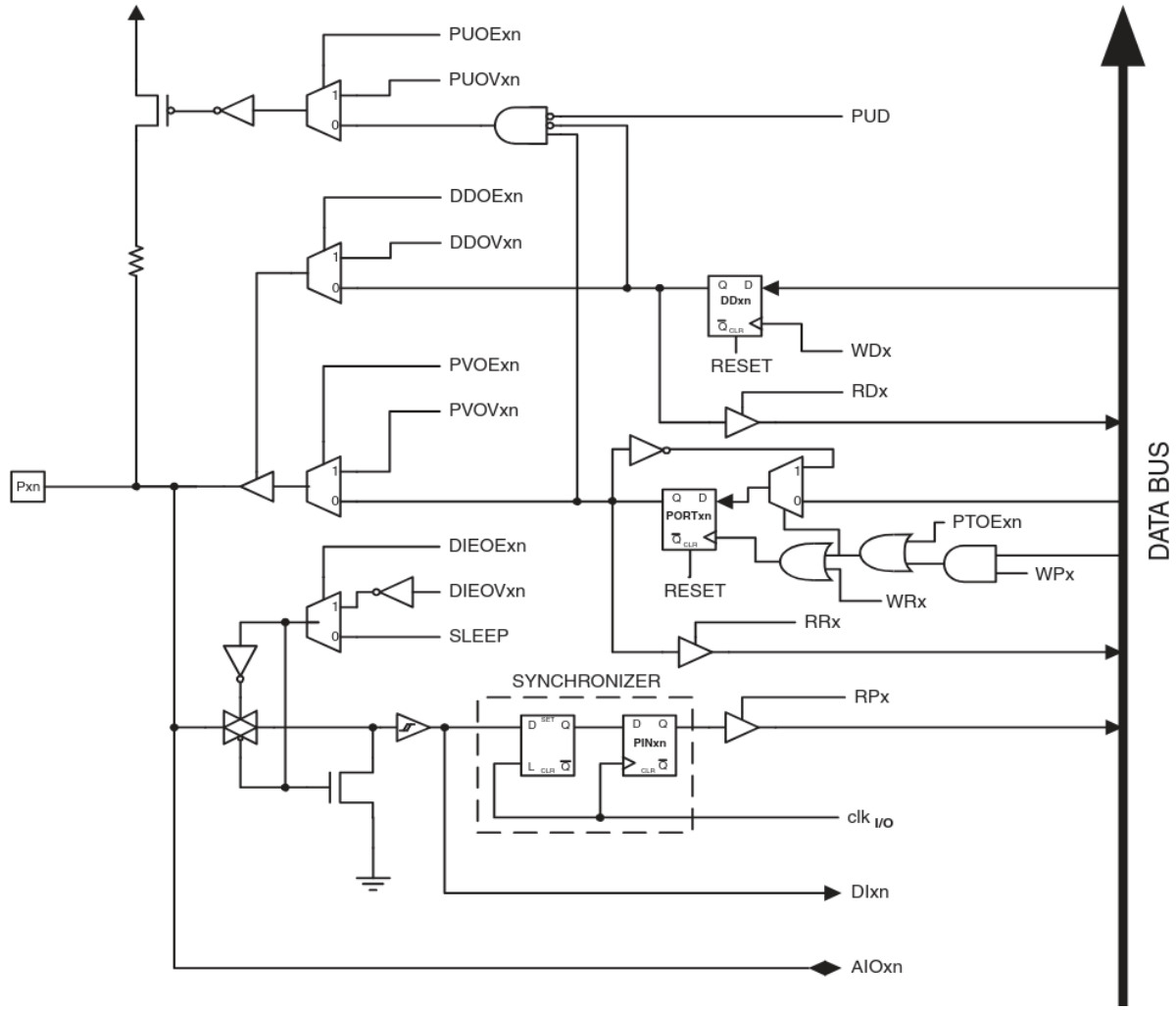
(c) Explain why PC + 2 is stored on the stack, and why the SP is decremented by 2.

Solution: First, storing PC on the stack is critical as it allows the program to pick up from where it left off after the function call is complete, therefore we need to store the program counter on the stack. If we stored the current program counter on the stack, we would carry out the same 'call' instruction and end up in an infinite loop, so we need to store the address of the instruction after this call instruction. Because the call instruction takes two words (with the 2nd word being comprised `k` bits), the next actual instruction is the instruction two words after the call instruction, hence why PC + 2 is stored on the stack.

The PC is a 16-bit value, and therefore takes two bytes on the stack. This therefore means that the next free byte is two addresses lower on the stack than where stack pointer was prior to carrying out this instruction, as the stack is allocated from the largest possible address towards the smallest.

Q3.

Consider the alternate port function diagram shown in Figure 1.



PUOExn:	Pxn PULL-UP OVERRIDE ENABLE	PUD:	PULLUP DISABLE
PUOVxn:	Pxn PULL-UP OVERRIDE VALUE	WDx:	WRITE DDRx
DDOExn:	Pxn DATA DIRECTION OVERRIDE ENABLE	RDx:	READ DDRx
DDOVxn:	Pxn DATA DIRECTION OVERRIDE VALUE	RRx:	READ PORTx REGISTER
PVOExn:	Pxn PORT VALUE OVERRIDE ENABLE	WRx:	WRITE PORTx
PVOVxn:	Pxn PORT VALUE OVERRIDE VALUE	RPx:	READ PORTx PIN
DIEOExn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE ENABLE	WPx:	WRITE PINx
DIEOVxn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE VALUE	$clk_{I/O}$:	I/O CLOCK
SLEEP:	SLEEP CONTROL	DIxn:	DIGITAL INPUT PIN n ON PORTx
PTOExn:	Pxn, PORT TOGGLE OVERRIDE ENABLE	AIOxn:	ANALOG INPUT/OUTPUT PIN n ON PORTx

Figure 1: Diagram of the internal GPIO circuitry of the ATMega32u2, including alternate function circuitry.

(a) For the pin to be in output mode, explain what the tristate buffer closest to the pin will need to do, and what signal it needs on the `enable` connection to do this.

Solution: It needs to conduct, for which the `enable` signal will need to be `HIGH` / 1.

(b) Determine what value `PVOExn` will need to be in to allow the signal from `PORTxn` to be selected in the multiplexer connected to the above tristate buffer.

Solution: The `PORTxn` connection to the multiplexer is in to input number 0. Therefore, the select line of the multiplexer will need to be 0 to effectively connect the input number 0 to the output.

The select line of the multiplexer is `PVOExn`, so therefore `PVOExn` needs to be 0.

(c) Complete the remaining values required for the signals below to place the pin into standard GPIO operating mode, with a `HIGH` output.

Signal	State	Signal	State
PUOExn		PUOVxn	
DDOVxn		DDOExn	
DDxn		PVOExn	
PVOVxn		PORTxn	
(others)			0

Solution:

Signal	State	Signal	State
PUOExn	0	PUOVxn	0
DDOVxn	0	DDOExn	0
DDxn	1	PVOExn	0
PVOVxn	0	PORTxn	1
(others)			0

(d) Determine what values are required for signals in the below Table to place the pin into alternate function mode, configured as an input with the pullup enabled.

Signal	State	Signal	State
PUOExn		PUOVxn	
DDOVxn		DDOExn	
DDxn		PVOExn	
PVOVxn		PORTxn	
(others)			0

Solution:

Signal	State	Signal	State
PUOExn	1	PUOVxn	1
DDOVxn	0	DDOExn	1
DDxn	X	PVOExn	X
PVOVxn	X	PORTxn	X
(others)			0

Where X is 'do not care'

END OF QUESTIONS